

# D2: Divide & Conquer

**Divide and conquer  
algorithm when  
multiply and  
surrender walks in**



Sec 17: Fr 11:30am-12:30pm FXB 1024  
IA: Pearl Lin

# Agenda

- ▶ Admin
- ▶ Master Theorem
- ▶ Divide and Conquer
- ▶ Worksheet

# Admin

- ▶ Office hours change:
  - ▶ ~~Monday 4:30-6:30PM in Leinweber 2421~~
  - ▶ **Sunday 2-4PM in BBB 1637**
- ▶ HW 2 released, due next Wednesday 9/16 at 8PM
  - ▶ Reminder that lowest 2 HW scores are dropped
- ▶ Don't wait to submit the attendance form! It will close ~5 minutes after discussion ends

# Master Theorem

# Master Theorem

- Provides a closed form runtime solution for recurrence relations
  - Use to analyze runtime in divide and conquer algorithms (later)

Original:

For a recurrence relation of the form  $T(n) = kT\left(\frac{n}{b}\right) + O(n^d)$ , where  $k \geq 1, b > 1, d \geq 0$  are constants:

$$T(n) = \begin{cases} O(n^d), & k < b^d \\ O(n^d \log n), & k = b^d \\ O(n^{\log_b k}), & k > b^d \end{cases}$$

With Log Factors:

For a recurrence relation of the form  $T(n) = kT\left(\frac{n}{b}\right) + O(n^d (\log n)^w)$ , where  $k \geq 1, b > 1, d \geq 0, w \geq 0$  are constants:

$$T(n) = \begin{cases} O(n^d (\log n)^w), & k < b^d \\ O(n^d (\log n)^{w+1}), & k = b^d \\ O(n^{\log_b k}), & k > b^d \end{cases}$$

Notes:

Bounds are tight: if  $O$  is replaced with  $\Theta$  in  $T(n)$ , then replace  $O$  with  $\Theta$  in closed form runtime solution as well.

Original Master Theorem is just a special case of Master Theorem with Log Factors with  $w = 0$ .

# Divide and Conquer

# Intro

## ► Key ingredients:

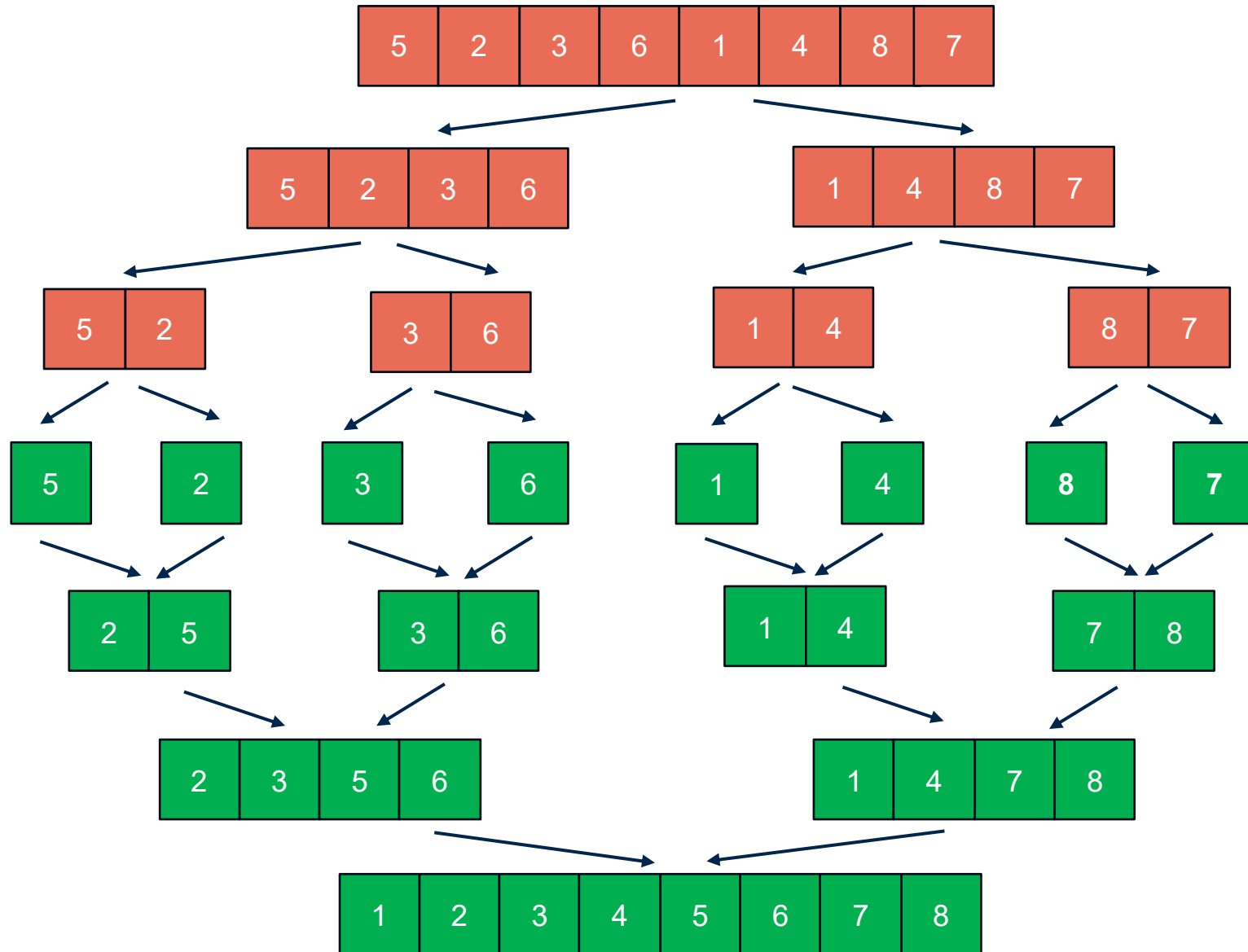
- **Divide**: split problem into smaller versions of the same problem
- **Conquer**: combine solutions from subproblems together
- **Base case(s)**: the smallest possible version(s) of the problem (usually trivial to solve)

Divide

Base case

Conquer

Example: Merge Sort



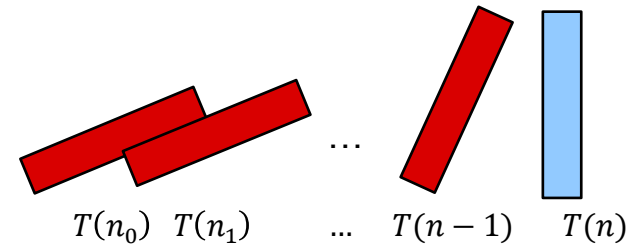
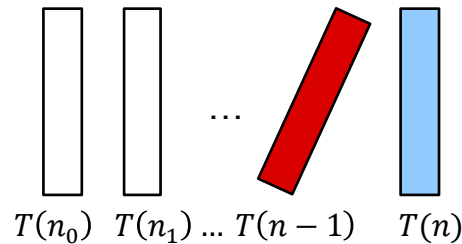
# Correctness Analysis

**Note:** Since the input size  $n$  in divide and conquer algorithms doesn't change by  $\pm 1$  each iteration like expected in proofs by induction, it may be necessary to redefine what parameter to use in the proof.

Define  $t = \log_b n$ , where  $\frac{n}{b}$  is the size of a subproblem. Then with each iteration of the algorithm,  $n$  changes by a factor of  $b$ . Since  $n = b^t$ ,  $\frac{n}{b} = b^{t-1}$  and  $bn = b^{t+1}$ . This leaves  $t$  to change by  $\pm 1$ , as desired.

- ▶ In most cases, **proof by induction** works well
  - ▶ Show that base case(s) are correct
  - ▶ Show that the method of combining subproblem solutions is correct (i.e., it gives the correct solution for original problem) through the inductive step
    - ▶ WTS:  $T(n)$  is correct assuming subproblem solution(s) are correct
    - ▶ If only the most immediate subproblem is used (i.e.,  $T(n-1)$ ), use **weak induction**
    - ▶ Otherwise, use **strong induction**

$T(n-1)$   
 $\rightarrow T(n)$



# Runtime Analysis: General Form

$k$ : # of recursive calls

► A divide and conquer algorithm breaks a problem of size  $n$  into:

►  $k$  smaller subproblems where  $k \geq 1$

► Each subproblem is of size  $\frac{n}{b}$ , where  $b > 1$

► The cost of splitting and combining results is  $O(n^d)$ , where  $d \geq 0$

► Recurrence relation can then be expressed as  $T(n) = kT\left(\frac{n}{b}\right) + O(n^d)$

► Looks familiar? You can use [Master Theorem](#) to get a closed form runtime solution

$b$ : input size of subproblem

$d$ : number of non-recursive work

# Worksheet

# Q1A

Given an array  $A[1, \dots, n]$  of  $n$  elements, the *majority element* of  $A$  is the element that appears strictly more than  $n/2$  times, if such an element exists. Note that if a majority element exists, then it is unique—so we say “*the* majority element” instead of “*a* majority element”. The algorithm MAJORITYELEMENT below outputs the majority element of  $A$  if it exists, otherwise it outputs a special symbol  $\perp$ . For simplicity we assume that  $n$  is a power of two, so that we can always split the array exactly in half (and don’t have to deal with annoying floors and ceilings).

- (a) Give a recurrence, and asymptotic closed-form solution, for the running time of MAJORITYELEMENT.

```
1: function MAJORITYELEMENT( $A[1, \dots, n]$ )
2:   if  $n = 1$  then return  $A[1]$ 
3:    $x \leftarrow$  MAJORITYELEMENT( $A[1, \dots, n/2]$ )
4:    $y \leftarrow$  MAJORITYELEMENT( $A[n/2 + 1, \dots, n]$ )
5:   if  $x \neq \perp$  then
6:     iterate over  $A$ , counting the number of occurrences of  $x$ 
7:     if the number of occurrences of  $x$  in  $A$  is  $> n/2$  then return  $x$ 
8:   if  $y \neq \perp$  then
9:     iterate over  $A$ , counting the number of occurrences of  $y$ 
10:    if the number of occurrences of  $y$  in  $A$  is  $> n/2$  then return  $y$ 
11:  return  $\perp$ 
```

# Q1A (annotations)

$k$ : # of recursive calls = 2

$b$ : subproblem input size = 2

$d$ : nonrecursive work = 1

- (a) Give a recurrence, and asymptotic closed-form solution, for the running time of MAJORITYELEMENT.

```
1: function MAJORITYELEMENT( $A[1, \dots, n]$ )
2:   if  $n = 1$  then return  $A[1]$ 
3:    $x \leftarrow$  MAJORITYELEMENT( $A[1, \dots, n/2]$ )
4:    $y \leftarrow$  MAJORITYELEMENT( $A[n/2 + 1, \dots, n]$ )
5:   if  $x \neq \perp$  then
6:     iterate over  $A$ , counting the number of occurrences of  $x$ 
7:     if the number of occurrences of  $x$  in  $A$  is  $> n/2$  then return  $x$ 
8:   if  $y \neq \perp$  then
9:     iterate over  $A$ , counting the number of occurrences of  $y$ 
10:    if the number of occurrences of  $y$  in  $A$  is  $> n/2$  then return  $y$ 
11:  return  $\perp$ 
```

$\frac{n}{2} \mid \leftarrow$   
 $\frac{n}{2} \mid \leftarrow$

$$\frac{k}{b^d} = \frac{2}{(2)^1} = 1$$

$\} O(n)$

$O(n \log n)$   $O(n^d \log n)$

## Q1A (solution)

Recurrence is in the form  $T(n) = k \cdot T\left(\frac{n}{b}\right) + O(n^d)$

$k$  = # of recursive calls = 2

$b$  = subproblem input size = 2 (arrays of size  $\frac{n}{2}$  are passed into recursive calls)

$d$  = non-recursive work = 1 (worst case iterate over A 2 times  $\Rightarrow O(n)$ )

Plugging into Master's Theorem:  $\frac{k}{b^d} = \frac{2}{2^1} = 1$ , so  $O(n \log n)$

- (a) Give a recurrence, and asymptotic closed-form solution, for the running time of MAJORITYELEMENT.

```
1: function MAJORITYELEMENT( $A[1, \dots, n]$ )
2:   if  $n = 1$  then return  $A[1]$ 
3:    $x \leftarrow$  MAJORITYELEMENT( $A[1, \dots, n/2]$ )
4:    $y \leftarrow$  MAJORITYELEMENT( $A[n/2 + 1, \dots, n]$ )
5:   if  $x \neq \perp$  then
6:     iterate over  $A$ , counting the number of occurrences of  $x$ 
7:     if the number of occurrences of  $x$  in  $A$  is  $> n/2$  then return  $x$ 
8:   if  $y \neq \perp$  then
9:     iterate over  $A$ , counting the number of occurrences of  $y$ 
10:    if the number of occurrences of  $y$  in  $A$  is  $> n/2$  then return  $y$ 
11:  return  $\perp$ 
```

$O(n)$

# Q1B

Hint: proof by contradiction works well here

- (b) Show the correctness of the MAJORITYELEMENT algorithm by first proving the following statement:

If  $z$  is the majority element of an array  $A[1, \dots, n]$ , then  $z$  must be the majority element of at least one of the subarrays  $A[1, \dots, n/2]$  and  $A[n/2 + 1, \dots, n]$ .

```
1: function MAJORITYELEMENT( $A[1, \dots, n]$ )
2:   if  $n = 1$  then return  $A[1]$ 
3:    $x \leftarrow$  MAJORITYELEMENT( $A[1, \dots, n/2]$ )
4:    $y \leftarrow$  MAJORITYELEMENT( $A[n/2 + 1, \dots, n]$ )
5:   if  $x \neq \perp$  then
6:     iterate over  $A$ , counting the number of occurrences of  $x$ 
7:     if the number of occurrences of  $x$  in  $A$  is  $> n/2$  then return  $x$ 
8:   if  $y \neq \perp$  then
9:     iterate over  $A$ , counting the number of occurrences of  $y$ 
10:    if the number of occurrences of  $y$  in  $A$  is  $> n/2$  then return  $y$ 
11:  return  $\perp$ 
```

# Q1B (annotations)

if  $P \rightarrow Q$

$P \wedge \neg Q$

(b) Show the correctness of the MAJORITYELEMENT algorithm by first proving the following statement:

If  $z$  is the majority element of an array  $A[1, \dots, n]$ , then  $z$  must be the majority element of at least one of the subarrays  $A[1, \dots, n/2]$  and  $A[n/2 + 1, \dots, n]$ .

$\rightarrow$   $z$  is majority in  $A$ , and  
 $z$  is not majority in  $A_L$  and  $A_R$ .

```
1: function MAJORITYELEMENT( $A[1, \dots, n]$ )
2:   if  $n = 1$  then return  $A[1]$ 
3:    $x \leftarrow$  MAJORITYELEMENT( $A[1, \dots, n/2]$ )
4:    $y \leftarrow$  MAJORITYELEMENT( $A[n/2 + 1, \dots, n]$ )
5:   if  $x \neq \perp$  then
6:     iterate over  $A$ , counting the number of occurrences of  $x$ 
7:     if the number of occurrences of  $x$  in  $A$  is  $> n/2$  then return  $x$ 
8:   if  $y \neq \perp$  then
9:     iterate over  $A$ , counting the number of occurrences of  $y$ 
10:    if the number of occurrences of  $y$  in  $A$  is  $> n/2$  then return  $y$ 
11:  return  $\perp$ 
```

$\Rightarrow \# \text{ of } z\text{'s in } A > \frac{n}{2}$   
 $\Rightarrow \# \text{ of } z\text{'s in } A_L \leq \frac{n}{4}$   
 $\Rightarrow \# \text{ of } z\text{'s in } A_R \leq \frac{n}{4}$   
 $\Rightarrow \# \text{ of } z\text{'s} \leq \frac{n}{2}$

contradiction.

## Q1B (solution)

(b) Show the correctness of the MAJORITYELEMENT algorithm by first proving the following statement:

If  $z$  is the majority element of an array  $A[1, \dots, n]$ , then  $z$  must be the majority element of at least one of the subarrays  $A[1, \dots, n/2]$  and  $A[n/2 + 1, \dots, n]$ .

Suppose for a contradiction that  $z$  is the majority element in  $A[1, \dots, n]$  and  $z$  is not the majority element in both  $A[1, \dots, \frac{n}{2}]$  and  $A[\frac{n}{2} + 1, \dots, n]$ .

$$\# \text{ of } z\text{'s in } A[1, \dots, \frac{n}{2}] \leq \frac{\frac{n}{2}}{2} = \frac{n}{4}$$

$$\# \text{ of } z\text{'s in } A[\frac{n}{2} + 1, \dots, n] \leq \frac{\frac{n}{2}}{2} = \frac{n}{4}$$

$$\Rightarrow \# \text{ of } z\text{'s in } A \leq \frac{n}{4} + \frac{n}{4} = \frac{n}{2}. \text{ Contradiction: } z \text{ is the majority element of } A, \text{ thus its count must be } > \frac{n}{2}$$

Rest of proof (sketch):

Inductive step claim: If  $x, y$  are the respective majority elements of  $A[1, \dots, \frac{n}{2}]$  and  $A[\frac{n}{2} + 1, \dots, n]$ , then algorithm returns the correct majority element for the entire  $A$ .

By the given fact, majority element if it exists must be either  $x$  or  $y$ . Indeed, the algorithm checks both  $x$  and  $y$  to see if it is the majority and returns it if it is. If not, the majority element doesn't exist and the algorithm correctly returns the null result.

Base case ( $n = 1$ ): array of size 1 has a majority element  $A[1]$  that appears  $n > \frac{n}{2}$  times

# Q2

## 3. Complete Binary Tree Local Max

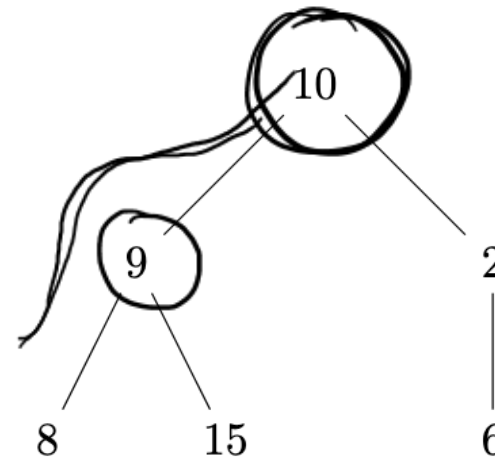
A *complete* binary tree is a binary tree in which every level, except possibly the last (leaf) level, is completely filled, and all the nodes in the last level are as far left as possible.

Consider a complete binary tree  $T = (V, E, r)$  rooted at  $r$  where each vertex is labelled with an integer. A vertex  $v \in V$  is called a *local maximum* if the label of  $v$  is greater than or equal to the labels of all its adjacent nodes—namely, its parent and children (whichever ones exist).

Hint: always move down towards an element that can be a local max

How many vertices do you have to cut from consideration each time to achieve the desired runtime?

can only go down 1 subtree  
→ low R



In this example, the local maxima are 10, 15, and 6.

Suppose you are given such a tree where the labelling is given implicitly, i.e., the only way to determine the label of the vertex  $v$  is to visit  $v$  and query the label. Give an algorithm that computes a local maximum of  $T$  using just  $O(\log|V|)$  vertex-label queries. Note:  $T$  may contain multiple local maxima; the algorithm just needs to return one of them.

case 1: children 1 & 2  
 $\leq$  root

return root

case 2: at least 1

child is  $>$  root.

more down  
more child

## Q2 (annotations)

Consider a complete binary tree  $T = (V, E, r)$  rooted at  $r$  where each vertex is labelled with an integer. A vertex  $v \in V$  is called a *local maximum* if the label of  $v$  is greater than or equal to the labels of all its adjacent nodes—namely, its parent and children (whichever ones exist).

Suppose you are given such a tree where the labelling is given implicitly, i.e., the only way to determine the label of the vertex  $v$  is to visit  $v$  and query the label. Give an algorithm that computes a local maximum of  $T$  using just  $O(\log|V|)$  vertex-label queries. Note:  $T$  may contain multiple local maxima; the algorithm just needs to return one of them.

Case 1: return the root  
correctness is because root  $\geq$  both children, so  $r$  is a local max

Case 2: return a grandchild  $T'$   
correctness  $\square$  best

Case 2A:  $r \neq \text{localmax}(T')$  where  $r$  is not  
maxed.  
Case 2B:  $r \in \text{localmax}(T')$   
if the root

## Q2 (solution)

Consider a complete binary tree  $T = (V, E, r)$  rooted at  $r$  where each vertex is labelled with an integer. A vertex  $v \in V$  is called a *local maximum* if the label of  $v$  is greater than or equal to the labels of all its adjacent nodes—namely, its parent and children (whichever ones exist).

Suppose you are given such a tree where the labelling is given implicitly, i.e., the only way to determine the label of the vertex  $v$  is to visit  $v$  and query the label. Give an algorithm that computes a local maximum of  $T$  using just  $O(\log|V|)$  vertex-label queries. Note:  $T$  may contain multiple local maxima; the algorithm just needs to return one of them.

**Algorithm idea:** maintain invariant of always having the input tree's root be at least as large as its parent (if the parent exists)

**Proof of invariant (by induction):** The first call to LocalMax is with the original tree, whose root has no parent. Subsequent calls to LocalMax are only ever with a subtree  $T'$  that is rooted at a larger or equivalent child.

```
LOCALMAX( $T$ ):
```

```
    if root is at least as large as all children then return root
```

```
    let  $T'$  = subtree rooted at a child that is at least as large as root
```

```
    return LocalMax( $T'$ )
```

## Q2 (solution)

Consider a complete binary tree  $T = (V, E, r)$  rooted at  $r$  where each vertex is labelled with an integer. A vertex  $v \in V$  is called a *local maximum* if the label of  $v$  is greater than or equal to the labels of all its adjacent nodes—namely, its parent and children (whichever ones exist).

Suppose you are given such a tree where the labelling is given implicitly, i.e., the only way to determine the label of the vertex  $v$  is to visit  $v$  and query the label. Give an algorithm that computes a local maximum of  $T$  using just  $O(\log|V|)$  vertex-label queries. Note:  $T$  may contain multiple local maxima; the algorithm just needs to return one of them.

### Correctness Proof (sketch):

Inductive step claim: if LocalMax( $T'$ ) returns a valid local max of a left or right subtree of  $T$ , then the algorithm returns a valid local max of  $T$

Case 1: root( $T$ ) is at least as large as all children. Given the invariant, root( $T$ ) is also at least as large as its parent. The algorithm correctly returns root( $T$ ) as a local max.

Case 2: at least 1 child  $>$  root

Case 2A: LocalMax( $T'$ ) returns root( $T'$ ). By the inductive hypothesis, it is a local max of  $T'$ , meaning it is at least as large as its children (parent doesn't exist in  $T'$  subtree). However, it is also at least as large as its parent since root( $T'$ ) is a larger or equivalent child of root( $T$ ). So, the algorithm correctly returns it as a local max

Case 2B: LocalMax( $T'$ ) returns a vertex of  $T'$  that is not root( $T'$ ): By the inductive hypothesis, this vertex is a valid local max of  $T'$  and thus  $T$ . So, the algorithm correctly returns it as a local max

Base case: tree with only 1 element gets its root returned, no neighbors exist means root is automatically a local max

### Runtime analysis:

# of recursive calls = 1  $\Rightarrow k = 1$

Size of subproblem input =  $\frac{n}{2} \Rightarrow b = 2$

Nonrecursive work =  $O(1) \Rightarrow d = 0$

Plug into Master Theorem:  $\frac{k}{b^d} = 1$  so  $O(\log n)$

# Q3

## 3. Divide and Conquer for Integer Multiplication

Karatsuba's algorithm is a fast multiplication algorithm that computes the product of two positive integers  $x$  and  $y$  using three multiplications of smaller numbers, each with about half as many digits as  $x$  and  $y$ , plus some additions, subtractions, and digit shifts. In this question, we build on the idea by dividing the numbers into smaller pieces. Namely, we break each argument  $x, y$  into *three* equal-sized pieces, of one-third the size. (For simplicity, assume that the size of  $x$  and  $y$  is a power of three, which can be achieved by “padding” them with some leading zeros; this does not affect the asymptotics.)

Similar to Karatsuba's algorithm, a naive divide-and-conquer algorithm, which we call  $\mathcal{A}$ , can multiply  $x$  and  $y$  using **9 recursive multiplications** of numbers that are  $n/3$  digits long, along with some linear-time work to combine the results.

Using some clever algebraic manipulations, it is possible to devise a “super-Karatsuba” algorithm that uses only **5 recursive multiplications** of numbers that are about  $n/3$  digits long! As before, the results can still be combined in linear time. We call this Algorithm  $\mathcal{B}$ .

For each algorithm  $\mathcal{A}$  and  $\mathcal{B}$ , derive the recurrence relation for the time complexity  $T(n)$ , then find an asymptotic closed-form solution for  $T(n)$ . Which of the algorithms  $\mathcal{A}$  and  $\mathcal{B}$  is asymptotically faster? How do they compare (asymptotically) to Karatsuba's Algorithm?

# Q3 (annotations)

## 3. Divide and Conquer for Integer Multiplication

Karatsuba's algorithm is a fast multiplication algorithm that computes the product of two positive integers  $x$  and  $y$  using three multiplications of smaller numbers, each with about half as many digits as  $x$  and  $y$ , plus some additions, subtractions, and digit shifts. In this question, we build on the idea by dividing the numbers into smaller pieces. Namely, we break each argument  $x, y$  into *three* equal-sized pieces, of one-third the size. (For simplicity, assume that the size of  $x$  and  $y$  is a power of three, which can be achieved by “padding” them with some leading zeros; this does not affect the asymptotics.)

Similar to Karatsuba's algorithm, a naive divide-and-conquer algorithm, which we call  $\mathcal{A}$ , can multiply  $x$  and  $y$  using **9 recursive multiplications** of numbers that are  $n/3$  digits long, along with some linear-time work to combine the results.

Using some clever algebraic manipulations, it is possible to devise a “super-Karatsuba” algorithm that uses only **5 recursive multiplications** of numbers that are about  $n/3$  digits long! As before, the results can still be combined in linear time. We call this Algorithm  $\mathcal{B}$ .

For each algorithm  $\mathcal{A}$  and  $\mathcal{B}$ , derive the recurrence relation for the time complexity  $T(n)$ , then find an asymptotic closed-form solution for  $T(n)$ . Which of the algorithms  $\mathcal{A}$  and  $\mathcal{B}$  is asymptotically faster? How do they compare (asymptotically) to Karatsuba's Algorithm?

# Q3 (solution)

Recurrence is in the form  $T(n) = k \cdot T\left(\frac{n}{b}\right) + O(n^d)$

For A:

$k$  = # of recursive calls = 9

$b$  = subproblem input size = 3 (numbers with  $\frac{n}{3}$  digits are passed into recursive calls)

$d$  = non-recursive work = 1 (linear time to combine results  $\Rightarrow O(n)$ )

Plugging into Master's Theorem:  $\frac{k}{b^d} = \frac{9}{3^1} = 3$ , so  $O(n^{\log_3(9)})$ . This is asymptotically slower than original Karatsuba.

For B:

$k$  = # of recursive calls = 5

$b$  = subproblem input size = 3 (numbers with  $\frac{n}{3}$  digits are passed into recursive calls)

$d$  = non-recursive work = 1 (linear time to combine results  $\Rightarrow O(n)$ )

Plugging into Master's Theorem:  $\frac{k}{b^d} = \frac{5}{3^1} > 1$ , so  $O(n^{\log_3(5)}) \approx O(n^{1.464974})$ . This is asymptotically faster than original Karatsuba

## 3. Divide and Conquer for Integer Multiplication

Karatsuba's algorithm is a fast multiplication algorithm that computes the product of two positive integers  $x$  and  $y$  using three multiplications of smaller numbers, each with about half as many digits as  $x$  and  $y$ , plus some additions, subtractions, and digit shifts. In this question, we build on the idea by dividing the numbers into smaller pieces. Namely, we break each argument  $x, y$  into *three* equal-sized pieces, of one-third the size. (For simplicity, assume that the size of  $x$  and  $y$  is a power of three, which can be achieved by "padding" them with some leading zeros; this does not affect the asymptotics.)

Similar to Karatsuba's algorithm, a naive divide-and-conquer algorithm, which we call  $\mathcal{A}$ , can multiply  $x$  and  $y$  using **9 recursive multiplications** of numbers that are  $n/3$  digits long, along with some **linear-time** work to combine the results.

Using some clever algebraic manipulations, it is possible to devise a "super-Karatsuba" algorithm that uses only **5 recursive multiplications** of numbers that are about  $n/3$  digits long! As before, the results can still be combined in **linear time**. We call this Algorithm  $\mathcal{B}$ .

For each algorithm  $\mathcal{A}$  and  $\mathcal{B}$ , derive the recurrence relation for the time complexity  $T(n)$ , then find an asymptotic closed-form solution for  $T(n)$ . Which of the algorithms  $\mathcal{A}$  and  $\mathcal{B}$  is asymptotically faster? How do they compare (asymptotically) to Karatsuba's Algorithm?

# Additional Slides

# Master Theorem Intuition

$$T(n) = \begin{cases} O(n^d), & k < b^d \\ O(n^d \log n), & k = b^d \\ O(n^{\log_b k}), & k > b^d \end{cases}$$

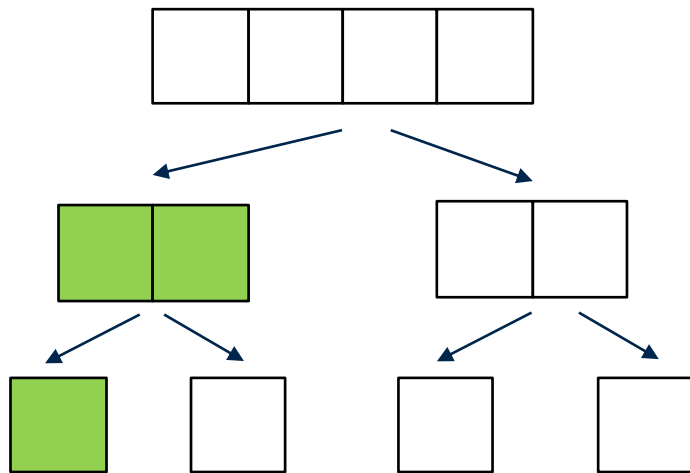
**Non-recursive** work dominates

**Balanced**

**Recursive** work dominates

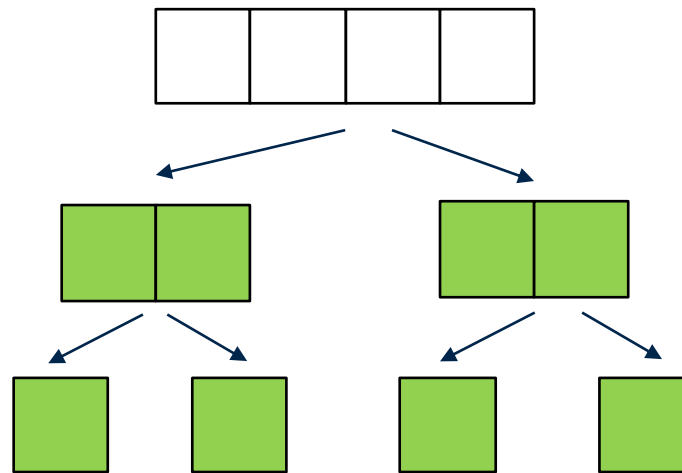
**Non-recursive** work dominates:  
solution tends towards non-recursive component ( $n^d$ )

ex.  $k = 1, b = 2, d = 1$

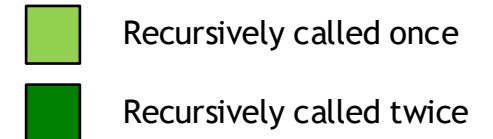


**Balanced:** solution is a mix of both recursive ( $\log n$ ) and non-recursive ( $n^d$ ) components

ex.  $k = 2, b = 2, d = 1$

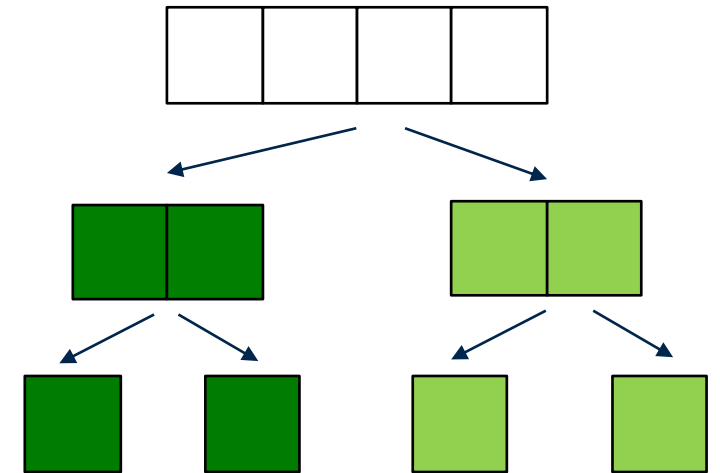


$$T(n) = \underbrace{kT\left(\frac{n}{b}\right)}_{\text{Recursive component}} + \underbrace{O(n^d)}_{\text{Non-recursive component}}$$



**Recursive** work dominates:  
solution tends towards recursive ( $\log n$ ) component

ex.  $k = 3, b = 2, d = 1$



# Karatsuba

- ▶ Big idea:

- ▶ Rewrite each integer as such:

- ▶  $X = A \left(10^{\frac{n}{2}}\right) + B$

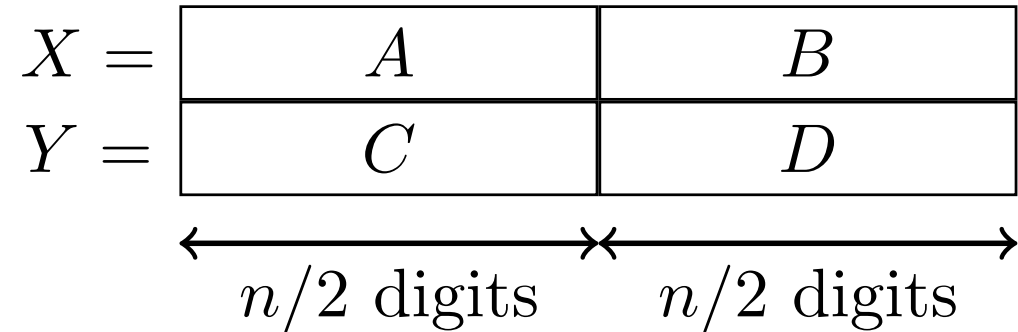
- ▶  $Y = C \left(10^{\frac{n}{2}}\right) + D$

- ▶ Expand the multiplication:

- ▶  $XY = \left(A \left(10^{\frac{n}{2}}\right) + B\right) \left(C \left(10^{\frac{n}{2}}\right) + D\right) = AC(10^n) + (AD + BC) \left(10^{\frac{n}{2}}\right) + BD$

- ▶ Observe that  $AC, AD, BC, BD$  are all multiplications of **smaller** integers (subproblems)

- ▶ This is a divide and conquer algorithm: we have split each integer into 2 smaller integers (**divide**) and used their products to get a solution to the original integer multiplication (**conquer**)



# Karatsuba (continued)

- ▶ Making things more efficient:
  - ▶ Carefully select what sub-multiplications to perform
    - ▶  $M_1 = AC$
    - ▶  $M_2 = BD$
    - ▶  $M_3 = (A + B)(C + D)$
  - ▶ Rewrite the recurrence:
    - ▶ By algebra,  $M_3 - M_1 - M_2 = AD + BC$
    - ▶  $XY = M_1(10^n) + (M_3 - M_1 - M_2)\left(10^{\frac{n}{2}}\right) + M_2$
    - ▶ Now we only need to do 3 sub-multiplications instead of 4!
    - ▶ Define base case to be single digit integers, just return their product
  - ▶ Runtime is  $O(n^{\log_2 3}) \approx O(n^{1.585})$ , better than  $O(n^2)$  for 4 sub-multiplications

```
KARATSUBA(x,y): //Assume  $x \geq y$   
  
     $n \leftarrow$  number of digits of  $x$   
    if  $n = 1$  then return  $x \cdot y$   
    write  $x$  as  $a \cdot 10^{n/2} + b$   
    write  $y$  as  $c \cdot 10^{n/2} + d$   
     $M_1 \leftarrow$  Karatsuba( $a, c$ )  
     $M_2 \leftarrow$  Karatsuba( $b, d$ )  
     $M_3 \leftarrow$  Karatsuba( $a + b, c + d$ )  
    return  $M_1 \cdot 10^n + (M_3 - M_1 - M_2) \cdot 10^{n/2} + M_2$ 
```

# Karatsuba Correctness/Runtime

## ► Correctness

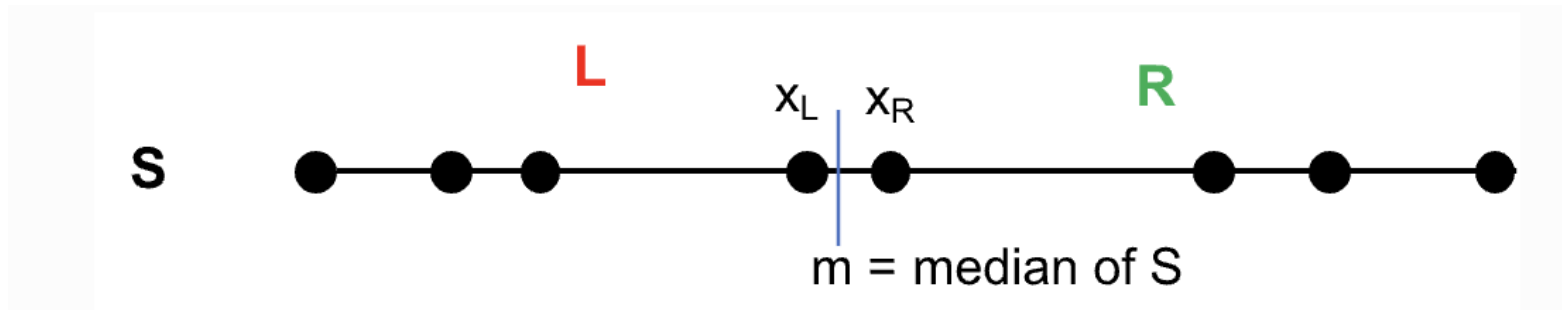
- Base case: correct since multiplication for single-digit numbers doesn't involve any other digits, it is simply the product of whatever digit they have
- Inductive step: Define  $t = \log_2 n$ . Assume  $P(t - 1)$ , i.e., multiplication of integers of size  $\frac{n}{2}$  is correct. WTS  $P(t)$ , i.e., multiplication of integers of  $X, Y$  of size  $n$  is correct.
  - So  $M_1 = AC$ ,  $M_2 = BD$ ,  $M_3 = (A + B)(C + D)$  since multiplication of integers of size  $\frac{n}{2}$  is correct
  - By algebra,  $M_1(10^n) + (M_3 - M_1 - M_2)\left(10^{\frac{n}{2}}\right) + M_2 = AC(10^n) + (AD + BC)\left(10^{\frac{n}{2}}\right) + BD = (A\left(10^{\frac{n}{2}}\right) + B)(C\left(10^{\frac{n}{2}}\right) + D)$
  - $= XY$ , as expected. So indeed  $P(t)$ .

## ► Runtime

- 3 recursive calls made on inputs of size  $\frac{n}{2}$
- Non-recursive work of adding and shifting is proportional to # of digits  $\rightarrow O(n)$
- $T(n) = 3T\left(\frac{n}{2}\right) + O(n)$ ,  $k = 3$ ,  $b = 2$ ,  $n = 1$
- $T(n) = O(n^{\log_2 3}) \approx O(n^{1.585})$ , by Master Theorem

# 1D Closest Pair

- ▶ Big idea:
  - ▶ Sort numbers and split them into 2 halves by their median
  - ▶ **Exactly one** of the following must be true for **any** pair:
    - ▶ Both points are in left half
    - ▶ Both points are in the right half
    - ▶ One point is in the left and the other is in the right (crossing pair)



# 1D Closest Pair (continued)

- ▶ **Exactly one** of the following must be true for a **closest** pair:
  - ▶ Case 1: both points of the closest pair are in the left half
    - ▶ Then it must be equal to a closest pair in **L**
  - ▶ Case 2: both points of the closest pair are in the right half
    - ▶ Then it must be equal to a closest pair in **R**
  - ▶ Case 3: it is a crossing pair
    - ▶ Then it must consist specifically of largest point in **L** and the smallest point in **R** (otherwise, an even closer pair can be found - contradicting our original assumption that the pair is a closest pair)
- ▶ Among the 3 options, take the one that has the smallest distance
- ▶ Runtime is  $O(n \log n)$  assuming sorting happened just once in the beginning, so no need to resort each recursive call

```
CLOSESTPAIR1D( $A[1, \dots, n]$ ): //Assume A is pre-  
sorted  
  
    if  $n = 1$  then return nothing  
    if  $n = 2$  then return ( $A[1], A[2], |A[1] - A[2]|$ )  
    partition array by its median into L, R  
    ( $l, l', \delta_L$ )  $\leftarrow$  ClosestPair1D(L)  
    ( $r, r', \delta_R$ )  $\leftarrow$  ClosestPair1D(R)  
     $p \leftarrow$  largest element in L  
     $p' \leftarrow$  smallest element in R  
    return one of ( $l, l', \delta_L$ ), ( $r, r', \delta_R$ ), ( $p, p', |p - p'|$ ) that has the smallest distance
```

# 1D Closest Pair Correctness/Runtime

- ▶ Correctness:

- ▶ Base cases:

- ▶  $(n = 1)$ : correct since no pairs exist, so there is no closest pair
    - ▶  $(n = 2)$ : correct since only 1 pair exists, so it is by default the closest pair

- ▶ Inductive step: Define  $t = \log_2 n$ . Assume  $P(t - 1)$ , i.e., closest pair of arrays of size  $\frac{n}{2}$  is correct. WTS  $P(t)$ , i.e., closest pair of array of size  $A$  of size  $n$  is correct.

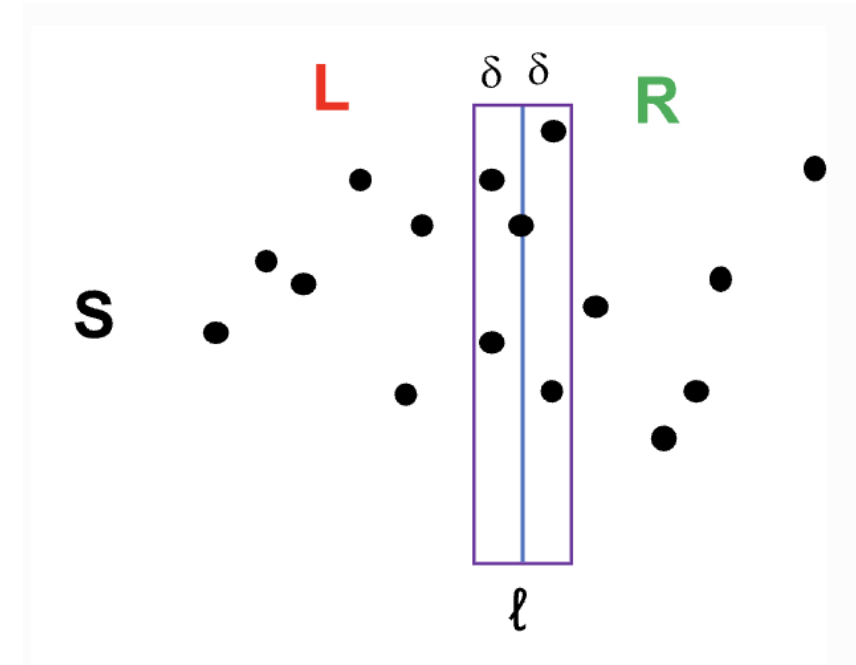
- ▶ So  $(l, l', \delta_L)$  is a closest pair in  $L$  and  $(r, r', \delta_R)$  is a closest pair in  $R$ , since closest pair of arrays of size  $\frac{n}{2}$  is correct
    - ▶ Consider a closest pair in  $A$ . Since it is a pair of elements in  $A$ , there are 3 cases:
      - ▶ Case 1: Both points are in  $L$ . Then a closest pair for  $A$  equals a closest pair for  $L$ . The algorithm returns either  $(l, l', \delta_L)$  or a pair of same distance, as desired. If the algorithm didn't, then  $(l, l', \delta_L)$  must not be a closest pair  $\rightarrow$  contradiction.
      - ▶ Case 2: Both points are in  $R$ . Then a closest pair for  $A$  equals a closest pair for  $R$ . The algorithm returns either  $(r, r', \delta_R)$  or a pair of same distance, as desired. If the algorithm didn't, then  $(r, r', \delta_R)$  must not be a closest pair  $\rightarrow$  contradiction.
      - ▶ Case 3: one point is in  $L$  and the other is in  $R$ . Then a closest pair  $A$  consists of the largest element in  $L$  and the smallest element in  $R$ . Otherwise, a closer pair can be formed, contradicting our original statement that the pair is a closest pair. Then the algorithm returns  $(p, p', |p - p'|)$  or a pair of same distance, as desired.
    - ▶ For all cases, a closest pair of  $A$  is returned. So indeed  $P(t)$ .

# 1D Closest Pair Correctness/Runtime (continued)

- ▶ Runtime (assuming presorted array)
  - ▶ 2 recursive calls made on inputs of size  $\frac{n}{2}$
  - ▶ Non-recursive work of copying into  $L$  and  $R$  takes  $\Theta(n)$  work
    - ▶ What if we don't copy arrays? What if we just pass the original array but specify the new start and end bounds? This would be  $\Theta(1)$  work and result in  $T(n) = \Theta(n)$  runtime from Master Theorem. But after adding on the extra runtime for presorting the array, the total runtime will still be the same because presorting will dominate
  - ▶  $T(n) = 2T\left(\frac{n}{2}\right) + \Theta(n)$ ,  $k = 2$ ,  $b = 2$ ,  $d = 1$
  - ▶  $T(n) = \Theta(n \log n)$
  - ▶ Don't forget we have to add on the runtime of presorting the array (take  $\Theta(n \log n)$  for merge sort)
  - ▶  $T'(n) = T(n) + \Theta(n \log n) = \Theta(n \log n)$

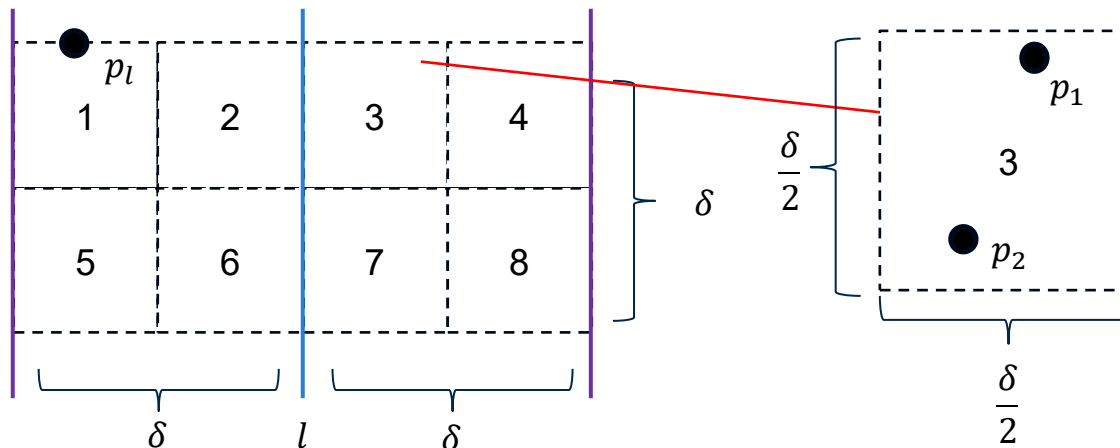
# 2D Closest Pair

- ▶ Big idea: use 1D closest pair method with x-coordinates, keeping in mind that we'll have to additionally contend with the extra dimension (y-coordinates) later
  - ▶ Sort numbers by x-coordinate and split them into 2 halves by median x-coordinate
  - ▶ Similarly, **exactly one** of the following must be true for a closest pair:
    - ▶ It is a closest pair in the left half ( $L$ )
    - ▶ It is a closest pair in the right half ( $R$ )
    - ▶ It is a crossing pair with the following condition
      - ▶ x-coordinates of both points must be within distance  $\delta = \min(\delta_L, \delta_R)$  of the median line
      - ▶ Otherwise, it cannot be a closest pair (a closest pair from  $L$  or  $R$  would have smaller distance)



# 2D Closest Pair (continued)

- ▶ It's still possible for ALL points to be in the  $\delta$ -strip in the crossing pair case! How do we narrow down the points we need to check?
  - ▶ Idea: For each point  $p_l$  in the  $\delta$ -strip, consider only points in the  $\delta$ -strip whose y-coordinate (without loss of generality) is greater and within  $\delta$  distance of  $p_l$ 
    - ▶ Candidate points reside in a  $2\delta$  by  $\delta$  box
    - ▶ Claim: Any point that creates a closest pair with  $p_l$  will reside in this box
      - ▶ Consider a point  $p_r$  that creates a closest pair with  $p_l$ . Then  $\text{dist}(p_l, p_r) \leq \delta$ . So  $|y_{p_l} - y_{p_r}| \leq \delta$ . Indeed,  $p_r$  resides in the box.
    - ▶ Claim: There are no more than 8 points (including  $p_l$ ) residing in this box. Specifically, if we divide the box into 8 squares as follows, each square cannot hold more than 1 element

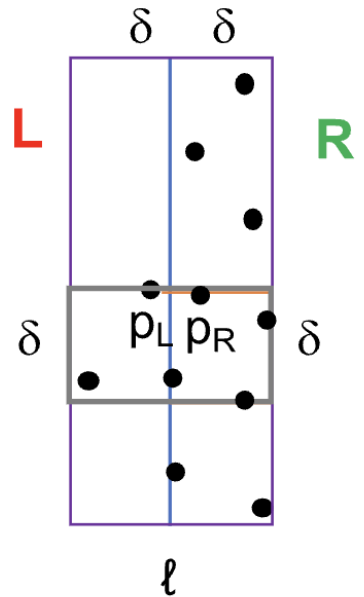


Suppose for a contradiction that a square holds more than 1 element.

So there exists  $p_1, p_2$  on  $R$  (WLOG) such that

$$\text{dist}(p_1, p_2) \leq \sqrt{\left(\frac{\delta}{2}\right)^2 + \left(\frac{\delta}{2}\right)^2} = \frac{\delta}{\sqrt{2}} < \delta.$$

**Contradiction:** There is a pair in  $R$  that has a distance less than  $\delta = \min(\delta_L, \delta_R)$ . So  $\delta_R$  cannot be the distance of the closest pair in  $R$ .



# 2D Closest Pair (continued)

- ▶ Main takeaway: for each point in the  $\delta$ -strip, we would only need to check at maximum a **constant** # of elements  $\rightarrow \Theta(n)$  if we presort the array by y coordinate
  - ▶ If we also presort the array by x coordinate (store in separate arrays), partitioning takes  $\Theta(n)$  time like in the 1D case
  - ▶ Total  $\Theta(n)$  time for non-recursive work
- ▶ Recurrence is  $T(n) = 2T\left(\frac{n}{2}\right) + \Theta(n)$ 
  - ▶  $k = 2, n = 2, d = 1$
  - ▶ So  $T(n) = \Theta(n \log n)$  by Master Theorem (same as 1D case!)
    - ▶ Adding on extra runtime for sorting doesn't change solution

```
CLOSESTPAIR2D(A[1, ..., n]): //Assume A is pre-
sorted by both x and y coordinate

    if n = 1 then return nothing
    if n = 2 then return (A[1], A[2], |A[1] - A[2]|)

    partition array by median x coordinate
    into L, R

    (l, l',  $\delta_L$ )  $\leftarrow$  ClosestPair2D(L)
    (r, r',  $\delta_R$ )  $\leftarrow$  ClosestPair2D(R)

     $\delta \leftarrow \min(\delta_L, \delta_R)$ 

    D  $\leftarrow$  set of points whose x coordinate
    are within  $\delta$  of median x coordinate, sorted
    by y coordinate

    for all p in D do:

        Consider the triple (p, p', |p - p'|) for
        up to 7 points preceding p in D

    return one of the triples among (l, l',  $\delta_L$ ),
    (r, r',  $\delta_R$ ), and those above that has smallest
    distance
```